

FINAL YEAR PROJECT · PROJECT REVIEW

Real-Time ECG Heart Visualizer

A single-lead electrocardiograph whose output is not a chart but an anatomically accurate 3-D human heart that contracts on every detected R-peak, in sync with synthesised heart sounds, above a live scrolling waveform.



● SYSTEM LIVE

1000 HZ

AD8232 · ARDUINO UNO R3

● AWAITING HARDWARE

Deployed at <https://143-198-27-18.nip.io>

An ECG trace is a diagnostic instrument. It is a poor teaching one.

A conventional monitor shows a line. To read it you must already know what the P wave, the QRS complex and the T wave *are*, and you must translate that line into an event happening inside a body you cannot see.

That translation is exactly the step a student has not yet made. The waveform is the abstraction; the beating heart is the thing.

The proposition: keep the clinical rigour of a real signal chain, and put the anatomy on screen as the primary output rather than the footnote.

Clinically honest

The same filtering and detection algorithm hospital monitors use, not a peak-finder.

Anatomically real

Four chambers, great vessels and coronary arteries. Not a symbol.

Physiologically timed

Atrial systole precedes ventricular systole, as in life.

Accessible

Runs in any browser, on any machine, with or without a network.

What it looks like



One abstraction holds the whole system together

Every data source emits the identical thing: **raw 10-bit ADC counts at 1000 Hz**, exactly what `analogRead(A0)` returns. Nothing downstream ever learns which one is connected.

SOURCE

Simulated AD8232, real board over USB, or samples forwarded from a browser

FILTER

0.5 Hz high-pass → 50/100 Hz notch → 40 Hz low-pass, stateful across chunks

DETECT

Streaming Pan-Tompkins with an adaptive threshold

CHANNEL

Per-client bounded queue, so a slow viewer never stalls acquisition

RENDER

3-D heart, waveform, heart sounds, CSV recorder

Why it matters

Swapping simulation for real hardware is a configuration change, not a rewrite. The downstream path has been exercised for hours before the board ever arrives.

Three implementations

`SimulatedSource`, `SerialSource` and `ClientFedSource`. Each is a peer; none is a special case.

Backpressure

A struggling browser drops its own frames. Acquisition stays at a true 1000 Hz. **Display degrades; timing never lies.**

Why Pan-Tompkins, and not a threshold

A fixed voltage threshold is defeated three ways: by baseline wander as the patient breathes, by a tall T wave, and by the amplitude collapse when an electrode dries out.

Pan-Tompkins solves all three by detecting QRS *energy* in the 5–15 Hz band and adapting its threshold to the running signal and noise estimates. It is the algorithm real monitors use.

BAND-PASS

5–15 Hz

DERIVATIVE

5-point

SQUARE

rectify

INTEGRATE

150 ms

THRESHOLD

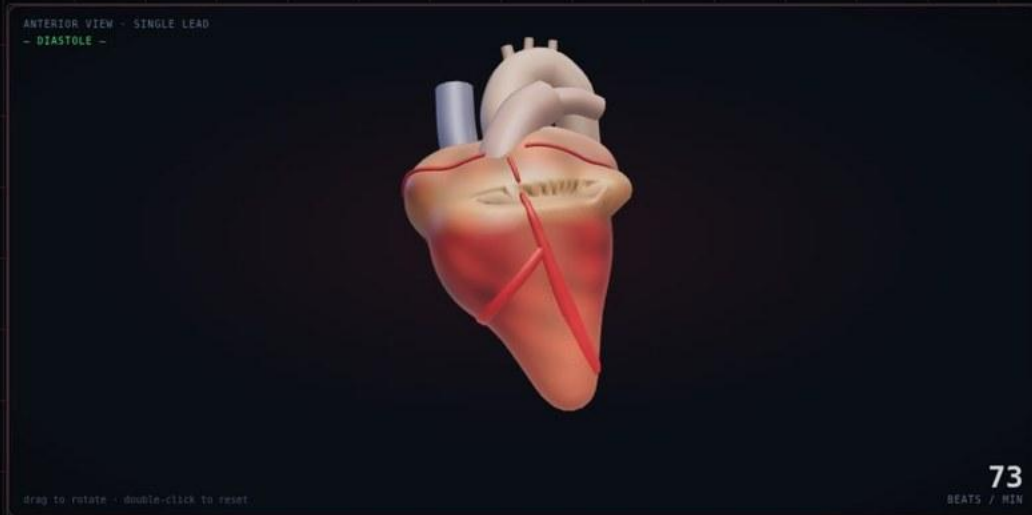
adaptive

Then it snaps back. The integrator's bump lags the true R-peak by about 100 ms, so the detection is refined against the filtered signal. Without that step the animation and the sound would both fire late.



Filtered trace (green) over the raw signal, with detected R-peaks marked.

Grown from mathematics, not downloaded



Built at startup from a signed distance field. 16k triangles, one draw call.

The heart is generated at runtime from a **signed distance field**: both ventricles, both atria, the auricles, the coronary sulcus and interventricular groove, the aortic arch with its three head vessels, the pulmonary trunk, both venae cavae, and the LAD, circumflex and RCA resampled onto the myocardial surface.

- ✓ Zero asset weight, no licensing, works offline
- ✓ One seamless organic surface, not a bag of primitives
- ✓ Occlusion, tissue type and contraction weights bake into vertex attributes
- ✓ The whole organ renders in **one draw call**; a beat costs four float uniforms

Orientation matters and is easy to get backwards: +x is the viewer's right and the *patient's left*, which puts the apex down to the viewer's right, which is what a true anterior view shows.

A cardiac cycle, not a pulse

Ventricular systole

Fires on the detected R-peak: radial squeeze, long-axis shortening, and apex-to-base counter-rotation, the wringing motion that separates a beating heart from a squeezing ball.

Atrial systole

Scheduled *ahead* of the next R-peak by predicting it from the running RR interval, because the P wave precedes the QRS by ~160 ms. The atria must contract before the ventricles, not after.

Ejection fraction

Blood volume follows a ~62% ejection fraction with rapid early filling, then diastasis. Systole scales with $\sqrt{RR}$, so at high rates it is *diastole* that vanishes, as in life.

S1 and S2

“Lub” is mitral and tricuspid closure, landing on the R-peak. “Dub” is aortic and pulmonic closure at the *end* of ejection, so it trails S1 by the systolic interval rather than a fixed gap.

Ordering is deliberate. A beat reaches the heart and the audio engine before any DOM work happens. Layout or style recalculation must never sit between the R-peak and what the user perceives.

Always on, reachable, and secure

The system runs permanently on an Ubuntu server under `systemd`, restarting on any exit. A single canonical address serves everything.

CONCERN	RESOLUTION
Uptime	<code>systemd</code> , <code>Restart=always</code> , survives reboot
TLS	Caddy + Let's Encrypt, auto-renewing
One link	bare IP and port 80 both 308-redirect to it
Mains hum	notch set to 50 Hz for Nigeria

HTTPS is not cosmetic here. Both headline features, reading a USB sensor from the browser and installing the app for offline use, are blocked outright outside a secure context.



The status bar: source, connectivity, measured latency, and the processing-mode switch.

20 ms

ROUND TRIP

0

DROPPED FRAMES

Three ways to run it, one codebase

Shared (online)

Everyone with the link sees the same session. Move a slider and every viewer's dashboard follows, controls included.

For demonstration and teaching.

Private (own sensor)

Asking for your own sensor steps out of the shared session entirely. Nobody else can alter what you see, and you alter nothing for them.

Because your trace is your heart.

Offline (installed)

Install the app and it runs entirely on your machine. No server, no network; the sensor is read directly.

For use where there is no connection.

A dashboard set to online still drops to local processing the instant the connection goes, and returns by itself, so **the trace never stops**. A deliberately chosen offline session is never dragged back by a reconnect.

How someone actually uses it

PATH A · SIMULATION, NO HARDWARE

- 1 Open the link** Nothing to install. The dashboard loads and joins the shared session.
- 2 Press Start Monitoring** The simulated AD8232 begins streaming at 1000 Hz.
- 3 Set a heart rate** Anything from 30 to 200 BPM. Add noise and motion artifacts to watch the filters work.
- 4 Watch and listen** The heart contracts on each R-peak. Press Sound for the lub-dub.
- 5 Record, then export CSV** Raw counts, filtered millivolts and flagged R-peaks, at full 1000 Hz resolution.

If no board is found, the trace stops. It does not quietly fall back to the simulation, so anything moving on screen in Path B is always a real reading.

PATH B · WITH A REAL ECG SENSOR

- 1 Open the secure link** Required. A browser will not open a USB port over an insecure connection.
- 2 Install the app (optional)** Unlocks offline mode, so the reading never depends on the network.
- 3 Wire the AD8232** Three electrodes on the chest, board to the computer by USB.
- 4 Press USB Sensor** The browser asks which board to use. Granted once, then remembered.
- 5 Press Arduino** The dashboard leaves the shared session. The trace on screen is now yours alone.
- 6 Record, then export CSV** Identical format to Path A, so the two are directly comparable.

The signal chain exists twice, and both agree exactly

Genuine offline operation is not a caching problem. A service worker can cache HTML and JavaScript; it cannot cache a running Python process. So the simulator, the filters and the detector had to exist **in the browser as well**.

Reimplementing filter *design* in JavaScript was rejected: it is fiddly numerical code whose failure mode is a trace that looks perfectly fine while reporting a different heart rate. Instead the SciPy filter coefficients are exported at build time, and only the twenty lines of arithmetic that *apply* them were ported.

The Python then acts as an **oracle**: both chains are fed identical samples and compared.

Identical raw ADC input, both chains

BPM	noise	filtered trace	beats
60	0.00	max diff 0.0e+00	10 = 10
60	0.30	max diff 0.0e+00	10 = 10
120	0.60	max diff 0.0e+00	21 = 21
150	1.00	max diff 0.0e+00	21 = 21
45	0.15	max diff 0.0e+00	8 = 8

R-peak index drift 0 samples

Reported BPM difference 0.000

Bit-exact, not merely close.

Offline and online cannot silently disagree about a heart rate, which is the one failure that would make the whole system untrustworthy.

What has actually been proven

20/20

SIGNAL-CHAIN CASES

86/87

BROWSER CHECKS

20/20

MULTI-CLIENT CHECKS

 ± 0.5

BPM ACCURACY

Signal-chain regression (the oracle)

BPM	noise	beats	measured	err	xRT
60	0.00	18	60.61	0.61	56x
60	1.00	18	59.94	0.06	55x
120	1.00	36	120.48	0.48	52x
150	1.00	45	149.63	0.37	50x
180	0.60	55	180.72	0.72	49x

All 20 cases passed.**xRT** = times faster than real time

Accuracy holds under abuse. Rates from 45 to 180 BPM, noise from clean to 100%, and step changes where the detector's learned thresholds are invalidated mid-recording.

Running roughly **50x faster than real time** means the backend sits near idle while streaming.

The browser suite drives the real application through a headless Chrome, including **cutting the network and reloading**, after which the app still boots and still detects beats.

Four decisions worth defending

Never block acquisition on a viewer

Sending directly to a socket let a slow browser apply TCP backpressure to the sampling loop. The UI drifted *seconds* into the past while still presenting stale data as live. On a monitor that is worse than useless.

Lost samples must not corrupt the rate

A dropout splices the waveform, and the interval spanning the seam reads short: one 0.4 s gap at 120 BPM turns a 500 ms RR into 100 ms. The detector now forgets its last peak, yielding *no* interval rather than a wrong one.

Measure before mitigating

The obvious fix for splices, long detection blanking, measured as a net negative. It discarded real beats to prevent phantoms that were not occurring. Blanking was cut to 120 ms.

A sensor is private by nature

Connecting a board once switched the *shared* source, stopping every other viewer's trace. Reading your own sensor now detaches you instead, and if none is found the trace stops rather than showing you someone else's.

Where the project stands

COMPLETE AND VERIFIED

- ✓ Signal chain: filtering, detection, simulation
- ✓ 3-D anatomy, cardiac cycle, heart sounds
- ✓ Always-on deployment with real TLS
- ✓ Offline operation and installable app
- ✓ Browser DSP proven identical to the server's
- ✓ Shared, private and offline session models
- ✓ Automatic updates for web and installed copies
- ✓ CSV export, latency metering, continuous integration

OUTSTANDING

- Validation against a physical AD8232 board
- Electrode placement and skin-prep procedure
- Comparison against a reference monitor

The honest position

Every path the hardware will take has been exercised for hours against a physically-modelled signal that includes gain, offset, quantisation, mains hum, motion artifacts and electrode dropout. **What remains is confirmation, not construction.**

Bringing the hardware in

The simulator models the AD8232 front end explicitly: $\times 1100$ gain, 1.5 V offset, 10-bit quantisation, Gaussian noise, mains hum and electrode-tug transients. A misbehaving real trace therefore points at physics, not at code.

STEP	WHAT IT ESTABLISHES	SUCCESS CRITERION
Bench bring-up	Board enumerates, sketch streams at 1000 Hz	Measured rate within 1% of 1000 Hz
Signal capture	Real PQRST morphology through the filter chain	P, QRS and T identifiable by eye
Rate agreement	Detector against a reference monitor	Within ± 2 BPM over 5 minutes
Artifact response	Leads-off and motion handling	Flagged, not silently mis-read
Latency	Electrode to animation, end to end	Under 150 ms

Beyond validation: multi-lead acquisition, arrhythmia classification, and session recording for later review are the natural extensions.

A working instrument, awaiting its sensor

1000 Hz

SAMPLE RATE

<150 ms

LATENCY BUDGET

16k

TRIANGLES, 1 DRAW CALL

0.0e+00

DSP DIVERGENCE

The project set out to make a heartbeat legible. What exists today is a verified signal chain, an anatomically faithful renderer, and a deployment that keeps working when the network does not.

Live system

<https://143-198-27-18.nip.io>

Source

github.com/holarxmanuel/ecg-heart-visualizer

Not a medical device. For education and demonstration only.